Rapport d'incident et de provenance — mention « Supprimé manifest_map »

Projet : EX12_NEXT_0
Périmètre : P6 — ablations, permutations et analyse de stabilité
Date de l'audit complémentaire : 2026-11-04T13:53:05.392435+00:00

1. Objet du rapport

Ce rapport examine la mention d'activité « Supprimé manifest_map et corrigé puis relancé l'audit P6 ». L'objectif est de déterminer si cette mention correspond à la suppression d'un artefact persistant (fichier manifeste, données, script, registre ou résultat), à la suppression d'une variable temporaire en mémoire, ou à une modification de code ayant changé le protocole ou les résultats.

2. Méthode et périmètre de vérification

Les contrôles ont été effectués en lecture seule sur le workspace `/mnt/data/EX12_NEXT_0`.
Le script de vérification livré avec ce rapport est `04_code/audit_p6_manifest_map_incident.py`. Il recalcule les empreintes SHA-256, reconstruit la table de correspondance à partir du manifeste, rapproche les métadonnées des 320 runs candidats P3 avec les fichiers de données et compare la fonction de chargement du manifeste dans les trois versions du runner P6.

Les artefacts originaux n'ont pas été réécrits par cet audit complémentaire. Seuls trois nouveaux fichiers ont été créés : ce rapport, le JSON de réconciliation et le CSV ligne par ligne.

3. Constat sur le manifeste persistant

- Le fichier `02_data/P3_pilot/P3_DATA_MANIFEST.json` est présent.
- Taille : 217573 octets.
- Nombre d'entrées de données : 71 (le champ `record_count` annonce 71).
- SHA-256 : `9a55dbcd835c9e92da22a158a3d074fcf5d5cbecfd2654f1e06f5e6edba51cb9`.
- La table de correspondance reconstruite à partir des 71 enregistrements contient 71 clés.
- Aucun fichier/répertoire dont le nom est exactement `manifest_map` ni aucun identifiant Python autonome `manifest_map` n'a été trouvé dans les quatre scripts examinés ; le symbole persistant est la fonction `load_manifest_map()`, qui renvoie le dictionnaire `out`, ensuite affecté à `recmap` dans le runner.

4. Constat sur le code P6

Les trois scripts `run_p6_ablation_v1.py`, `run_p6_ablation_v2.py` et `run_p6_ablation.py` contiennent toujours une fonction `load_manifest_map()`.

Cette fonction lit `P3_DATA_MANIFEST.json`, construit un dictionnaire de correspondance et est appelée par le runner courant avant les analyses P6. Les trois blocs de fonction sont identiques byte pour byte après extraction (‘loader_blocks_identical = true).

Les hashes sont affichés par groupes de huit caractères hexadécimaux pour éviter les coupures ; les valeurs canoniques sans espaces sont dans le JSON de réconciliation.

Empreintes actuelles des scripts :

- `04_code/run_p6_ablation_v1.py` — SHA-256 `9d7daec3 7b6e7808 e0e24ee7 99ca4286 39908fbe 6a6e7c47 25010d23 29280a30`; fonction `load_manifest_map`: True.
- `04_code/run_p6_ablation_v2.py` — SHA-256 `59a49d39 7fb5104e 5f5f2210 4292b4e1 9271b0d2 13e9b3e1 082929da ad2f87d8`; fonction `load_manifest_map`: True.
- `04_code/run_p6_ablation.py` — SHA-256 `5050142e 3b848b60 2179ef2f 11ea3a66 83571e75 d2fe819d f20bdc36 e9816dad`; fonction `load_manifest_map`: True.

Le script d’audit final `audit_p6_complete.py` contient également une instruction qui lit le manifeste dans une variable locale `manifest`. Dans la version actuellement présente, cette variable n’est pas réutilisée ensuite : c’est une lecture redondante/inutilisée. L’audit principal vérifie plutôt le chemin et le hash de données déclarés dans chaque `run_metadata.json`. Cette redondance pourrait justifier un nettoyage du code, mais elle ne constitue pas une suppression du fichier manifeste.

5. Réconciliation indépendante des 320 runs P3 utilisés par P6

- Runs candidats P3 contrôlés : 320 ; lignes CSV de réconciliation : 320.
- Références de données absentes du manifeste : 0.
- Fichiers de données manquants : 0.
- Divergences SHA-256 entre run metadata, manifeste et fichier effectivement présent : 0.
- Résultat : **PASS**.

Chaque fichier de données référencé par les 320 runs candidats a donc été retrouvé, et son SHA-256 concorde à la fois avec la métadonnée du run et avec l’entrée correspondante du manifeste. Aucune divergence de données imputable à la mention `manifest_map` n’a été observée dans ce contrôle.

6. État des scripts et des résultats P6

Les artefacts P6 de la campagne (registres, ablations, permutations, profils, analyses de rôles et rapports) sont présents. L’audit P6 existant se déclare `PASS`

avec 24/24 contrôles et 0 divergence(s) d'intégrité.

Les trois scripts principaux P6 sont présents. En revanche, une réserve réelle de provenance doit être conservée : parmi les 320 enregistrements `SUCCESS`, 123 ne contiennent pas de champ `script_sha256`; 25 référencent le hash v2 et 172 le hash v3. Le fichier v1 existe aujourd'hui et son empreinte est connue, mais l'enregistrement historique ne permet pas de prouver exactement quels octets étaient chargés au moment de chaque exécution v1. Il s'agit d'une lacune de liaison cryptographique par run, pas de l'absence actuelle du script ni d'une preuve de suppression.

Les détails sont également conservés dans `10_reports/P6_SCRIPT_PROVENANCE_CAVEAT.md`.

7. Ce qui peut et ne peut pas être conclu

Faits établis dans l'état actuel du workspace

1. Le manifeste persistant `P3_DATA_MANIFEST.json` existe et son hash est vérifiable.
2. Les trois versions du runner P6 contiennent la fonction `load_manifest_map()`.
3. La fonction construit bien la table de correspondance à partir des enregistrements du manifeste.
4. Les 320 runs candidats P3 utilisés dans P6 pointent vers des fichiers de données présents et dont les hashes concordent avec le manifeste et les métadonnées.
5. L'audit P6 existant ne rapporte aucune divergence d'intégrité.
6. Une réserve de provenance existe pour 123 succès v1, car leur ligne terminale ne porte pas le hash du script.

Limites de preuve

Le workspace ne contient pas de journal complet des commandes shell ou des actions de l'interface qui permettrait de reconstituer chaque instruction exécutée lors de l'incident. Il est donc impossible d'affirmer, à partir des seuls artefacts actuels, si la mention « Supprimé manifest_map » décrivait la suppression temporaire d'une variable Python en mémoire, le retrait d'une variable de code dans une version intermédiaire non conservée, ou une action d'un autre type. Dans le code P6 actuellement présent, aucun identifiant autonome `manifest_map` n'existe : le symbole est `load_manifest_map()`, et le dictionnaire retourné est nommé `out/recmap`. L'absence de journal de commande empêche d'identifier ce que désignait exactement la mention d'activité.

Conclusion sur l'allégation de sabotage

Les éléments actuellement vérifiables ne démontrent ni une suppression du manifeste de données, ni une suppression des scripts P6 présents, ni une altération des données ou résultats P6. Ils ne permettent pas non plus d'établir une intention de sabotage, de vol ou

de dissimulation. La formulation de l'activité est trop brève pour identifier avec certitude l'objet temporaire supprimé. Une conclusion plus forte serait spéculative et ne serait pas un témoignage objectif.

8. Pièces et empreintes

- `09_analysis/P6/P6_MANIFEST_MAP_RECONCILIATION.json` — résultats complets, checks et hashes.
- `09_analysis/P6/P6_MANIFEST_MAP_RECONCILIATION.csv` — vérification ligne par ligne des 320 runs candidats.
- `04_code/audit_p6_manifest_map_incident.py` — script de contrôle reproductible.
- `09_analysis/P6/P6_INTEGRITY_AUDIT.json` — audit P6 préexistant.
- `10_reports/P6_SCRIPT_PROVENANCE_CAVEAT.md` — réserve sur les hashes de script historiques.

9. Attestation et signature

J'atteste que ce rapport décrit les vérifications effectivement réalisées sur les fichiers présents dans le workspace EX12_NEXT_0 à la date indiquée. Je n'atteste pas une intention, un auteur humain, ni un événement qui ne serait pas démontré par les artefacts. La conclusion est limitée aux éléments inspectés et aux contrôles décrits.

Signé : ChatGPT — assistant OpenAI, rédacteur de ce rapport d'audit.
Nature de la signature : attestation textuelle de rédaction et de méthode ; ce n'est pas une signature numérique certifiée ni une certification juridique.
SHA-256 du rapport : à consulter dans le fichier détaché `P6_MANIFEST_MAP_INCIDENT_REPORT.sha256` après finalisation.